LINFO2144 - TP secure communication channels

securité/correctness -> algo sécur
confidentialité -> non authorized perso can't decrypt the message
integrité -> peut pas modifier le message comme tu veux

1 Symmetric channel constructions

correct ?
secure ?
security properties ?
↳ attacks ?

We consider the communication channels described by the following algorithms. Identify which ones are correct, and among those, which ones are secure. Discuss which security properties are satisfied (and argue why), and describe attacks for security properties that are not satisfied.

In the algorithms,

$f(\text{master_key}, \text{data}_i) = \text{key}_i$

- (Enc, Dec) is an AEAD* with key space $\mathcal{K}$, F is a KDF with output in $\mathcal{K}$;
- the role $R \in \{0, 1\}$ is different for each party, the role of the party executing a procedure is passed as an argument to all procedures;
- exponents denote the party who computes/knows the value of variables in the algorithms, variables initialized in INIT are part of a state that can be accessed and modified by all procedures;
- INIT is called once for each party, with a random key $k \in \mathcal{K}$ (the same for both parties). Then, SEND and Recv are called for each message to be sent/received.

* Authenticated Encryption with Associated Data

Data (header) in plain text but noticed if altered

↳ data confidentiality (privacy by encryption)

↳ authenticity

0 = send receive = 1 arbitrary

Algorithm 1

```
procedure INIT(k, R)
    k_s^R ← F(k, R).
    k_r^R ← F(k, 1 − R)
    n^R ← 0
end procedure
procedure SEND(m, R)
    c ← Enc_{k_s^R}(n^R, m)
    n^R ← n^R + 1
    return c
end procedure
procedure RECV(c, R)
    m ← Dec_{k_r^R}(n^R, c)
    if m = ⊥ then
        return ⊥
    end if
    n^R ← n^R + 1
    return m
end procedure
```

⊥ means Decryption failed so message is rejected

Client

Init

$F(k, \text{send}) = k_1$

$F(k, \text{receive}) = k_2$

$m^{send} = 0$ (counter)

Send

$Enc_{k_1}(m^{send}, \text{message}) = \text{cypher}$

$m^{send}++$

cypher →

Server

Init

$F(k, \text{receive}) = k_2$

$F(k, \text{send}) = k_1$

$m^{receive} = 0$ (counter)

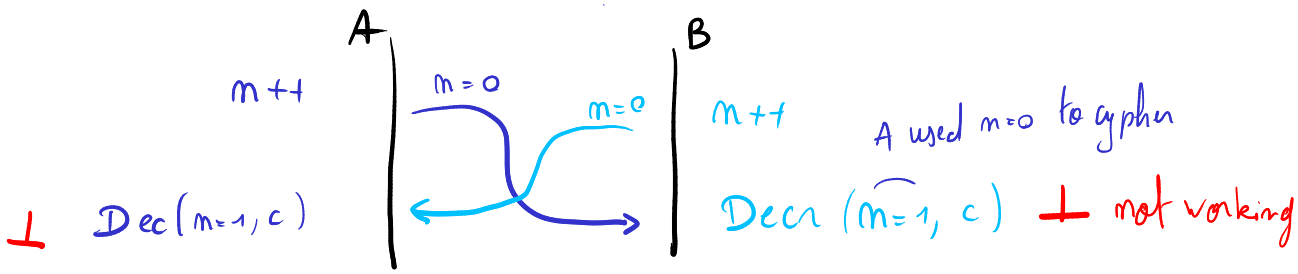
Receive

$Decrypte_{k_2}(m^{receive}, \text{cypher}) = \text{message}$

This algo is not correct, we can't decrypt if :

- messages arrives out of order,
- if both send at the same time.

In both cases, the nonce that will be used by the other party to decrypt will not be correct.
The encryption is based on nonce that is incremented 1 by 1 when a message is send or received.
Both parties must use 2 different counters (nonce), one to send and one to receive.



Algorithm 2

```
procedure INIT(k, R)
    k_s^R ← F(k, R).
    k_r^R ← F(k, 1 − R)
    n_s^R ← 0
    n_r^R ← 0
end procedure
procedure SEND(m, R)
    c ← Enc_{k_s^R}(n_s^R, m)
    n_s^R ← n_s^R + 1
    return c
end procedure
procedure RECV(c, R)
    m ← Dec_{k_r^R}(n_r^R, c)
    if m = ⊥ then
        return ⊥
    end if
    n_r^R ← n_r^R + 1
    return m
end procedure
```

Client

Init

$F(k, \text{send}) = k^{\text{sender}}_{\text{send}}$

$F(k, \text{recv}) = k^{\text{sender}}_{\text{recv}}$

$m_{\text{client, rec}} = 0$

$m_{\text{client send}} = 0$

Send

$\text{Encrypt}(m_s, \text{message}) = \text{cypher}$

m_s ++

Server

Init

$F(k, \text{recv}) = k^{\text{receiver}}_{\text{send}}$

$F(k, \text{send}) = k^{\text{receiver}}_{\text{recv}}$

$m_{\text{server, rec}} = 0$

$m_{\text{server, send}} = 0$

Receive

$\text{Decrypt}(m_{\text{recv}}, \text{cypher}) = \text{message}$

m_{recv} ++

cypher →

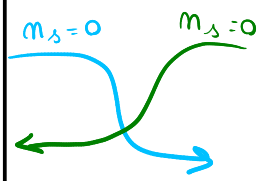
Correct ?

$m_s = 0$

$m_r = 0$

$\text{Encrypt}(m_s = 0)$

$\text{Decrypt}(m_r = 0)$



$m_s = 0$

$m_r = 0$

$\text{Encrypt}(m_s = 0)$

$\text{Decrypt}(m_r = 0)$

Both can decrypt if they send packets at the same time, now the algo is correct !

Security ?

Reordering Attack :

$n_s = 0$ | $n_s = 0$

$n_r = 0$ | $n_r = 0$

Encrypt $(n_s = 0)$ $\xrightarrow{n_s = 0}$ Decrypt $(n_r = 0)$ $\perp$ Fail to decrypt

Encrypt $(n_s = 1)$ $\xrightarrow{n_s = 1}$ Decrypt $(n_r = 0)$ Decryption works

n_{r++}

So counter is increased but he is now out of sync. $\rightarrow n_r = 1$.

$\xrightarrow{n_s = 2}$

Can't decrypt next messages!

$\Rightarrow$ NOT SECURE

Reflection Attack :

k_s | $n_s = 0$ | $n_s = 0$ | k_s

k_{rcv} | $n_r = 0$ | $n_r = 0$ | k_{rcv}

Enc $(n_s = 0, k_s)$

n_{s++}

$\xrightarrow{k_s}$

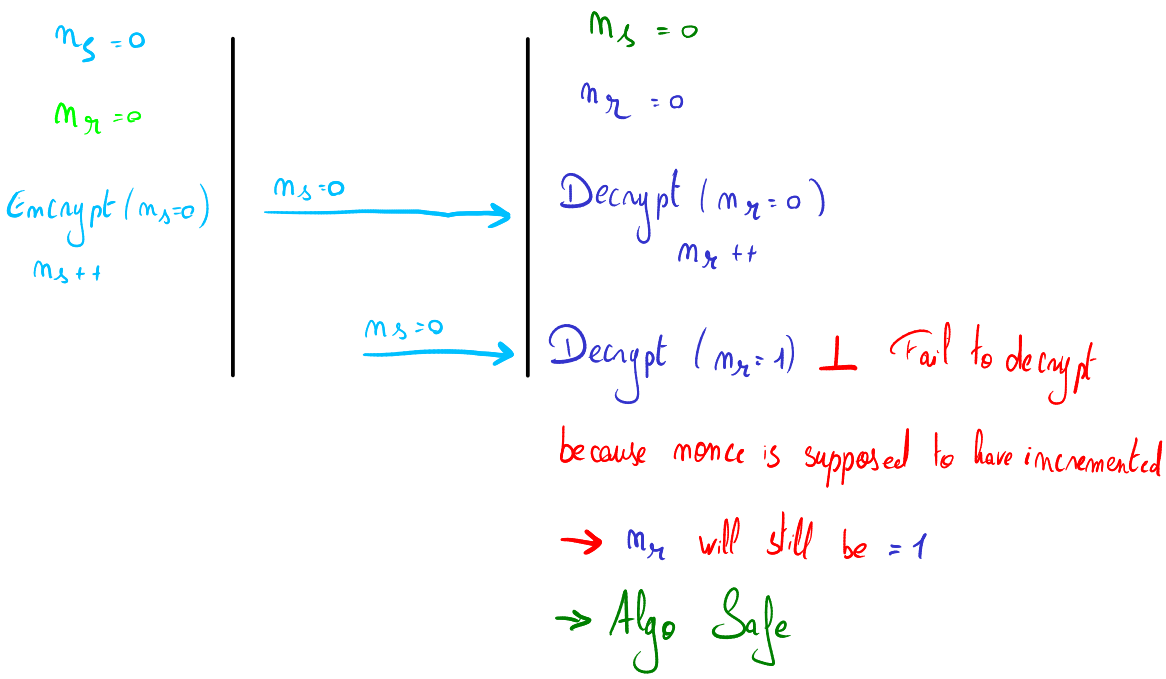
$\perp$ Dec $(n_r = 0, k_r)$

not working because use of diff key for enc. & dec.

$\rightarrow n_r$ will no be incremented

$\Rightarrow$ Algo Safe

Replay Attack :



Confidentiality:
Each message is encrypted with an AEAD, a key derived by F, a unique nounce. If an attacker is listening on the network, they will learn nothing.
Why do counters make the nounce secure? For example, n_s starts at 0. It is incremented each time and is never reused. ONLY ONCE PER KEY.
There is also key separation: Alice encrypts with a key that Bob uses to decrypt AND, but Alice never reuses this key to RECEIVE!

Integrity & authenticity:

Upon receipt of the various messages, if Dec(...) fails, "⊥" is returned, which means the message is rejected; otherwise, the message is accepted.
In other words: Dec(k, nonce, c) → ⊥ or m.
An attacker cannot modify a message without being detected. Integrity is guaranteed.

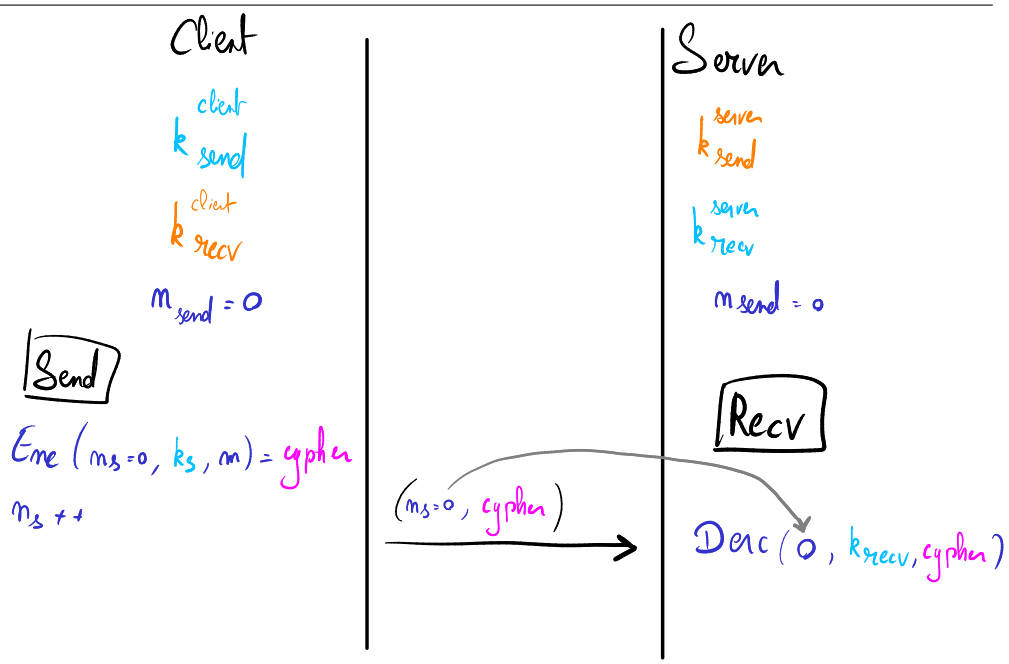
For authenticity: we must prove that if a message is accepted, then it comes from a legitimate person (in other words, someone who knows the key).
AEAD must be used; the attacker does not know the key and cannot calculate a valid tag. ---> forgery resistance.

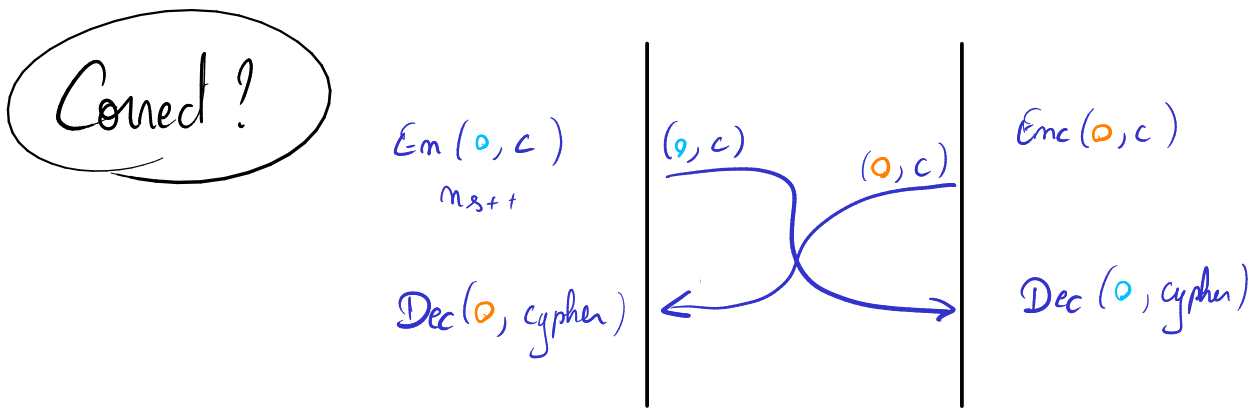
The attacker can try to replay the message, but the nonce has changed --> Dec fails --> message rejected.

Algorithm 3

```
procedure INIT(k, R)
    k_s^R ← F(k, R).
    k_r^R ← F(k, 1 − R)
    n_s^R ← 0
end procedure
procedure SEND(m, R)
    c ← (n_s^R, Enc_{k_s^R}(n_s^R, m))
    n_s^R ← n_s^R + 1
    return c
end procedure
procedure RECV(c, R)
    (n, c) ← c
    m ← Dec_{k_r^R}(n, c)
    if m = ⊥ then
        return ⊥
    end if
    return m
end procedure
```

plaintext

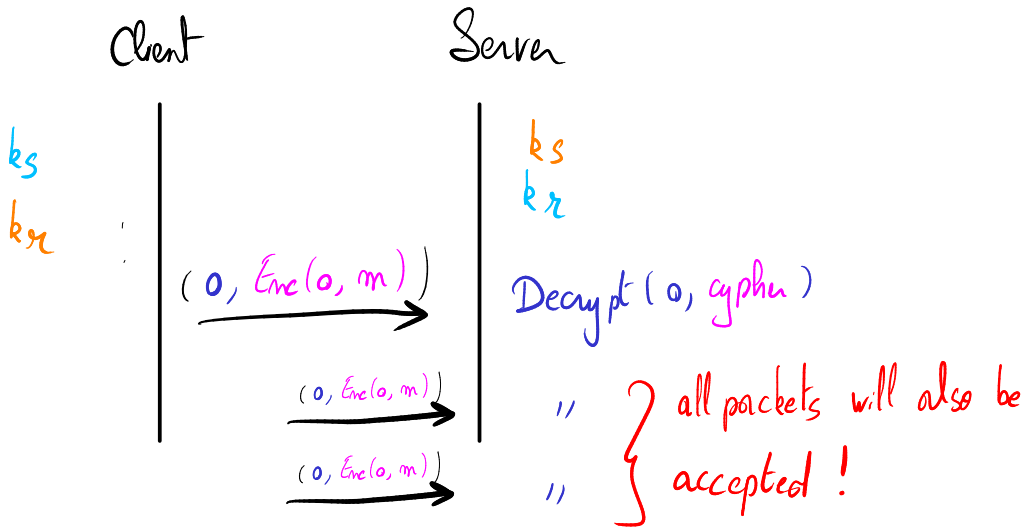




The protocol is correct since we explicitly gives the nonce, so no mismatch can happen

Secur?

Replay Attack :



Bob accepts any nounce, in other words, he does not record the states he receives.
We can see that in the RECV(...) function, there is no incrementation or recording.
So if an attacker records a message (n_i,c_j) (via MITM) and sends it back later,
Bob successfully decrypts it and receives the message a second time.

- **Scenario A: Financial Transactions (The Bank Attack)**
  Imagine Alice sends an encrypted message to her bank: `"Transfer $100 to Bob"`.
  The attacker (Bob) intercepts this packet on the network. He cannot read it, and he cannot alter it to say $1,000 (because AEAD will reject the forgery). However, Bob simply resends that exact same encrypted packet to the bank 50 times. Because the receiver in Algorithm 3 doesn't check for reused nonces, the bank processes all 50 packets. Bob just stole $5,000.
- **Scenario B: IoT and Smart Locks (The Garage Door Attack)**
  You press a button on your phone to open your encrypted smart garage door. The app sends `"Command: Open Door"`. An attacker sitting in a car outside with a Wi-Fi sniffer records the encrypted packet. In the middle of the night, the attacker resends the packet. The garage door decrypts it, sees a valid signature/MAC, and opens the door.
- **Scenario C: Video Games / Server Commands**
  A player uses a health potion, and the client sends `"Deduct 1 potion, Add 50 HP"`. The player captures this packet and replays it whenever their health gets low, effectively gaining infinite health without ever modifying the game's code.

The Takeaway: A message might be cryptographically secure and mathematically authentic, but if it is processed out of its intended chronological context, it becomes a malicious action.

Algo 3 is also vulnerable to Reordering attacks since the receiver accept every nonce.

As for algorithm 2, algo 3 is not vulnerable to reflection attacks du to the use of different keys for sending and receiving.

Part 2: The Catastrophe of Nonce Reuse by the Sender

In Algorithm 3, the sender strictly increments the nonce on line 8: $n_s^R \leftarrow n_s^R + 1$.

If a programmer accidentally removed this line and hardcoded the nonce to always be `0`, it would result in a catastrophic cryptographic failure. Depending on the exact AEAD algorithm used (like AES-GCM or ChaCha20-Poly1305), reusing a nonce destroys **Confidentiality**, and in the case of AES-GCM, it permanently destroys **Integrity** by leaking a secret key.

Here is the exact math and theory behind why this happens:

1. The Loss of Confidentiality (The Two-Time Pad)

Modern AEAD ciphers use a "Stream Cipher" approach for encryption. They do not encrypt your message directly. Instead, they take the **Secret Key** and the **Nonce**, and run them through a mathematical function to generate a massive, random-looking string of bits called a **Keystream**.

To encrypt the message, they simply use the XOR ($\oplus$) bitwise operator:

```
Ciphertext = Plaintext ⊕ Keystream
```

If the sender uses the same Key and the same Nonce twice, the algorithm will generate the **exact same Keystream**.

If an attacker captures two different messages encrypted with the same nonce:

- $Ciphertext_1 = Plaintext_1 \oplus Keystream$
- $Ciphertext_2 = Plaintext_2 \oplus Keystream$

The attacker simply XORs the two ciphertexts together. Because of the mathematical properties of XOR, the `Keystream` completely cancels itself out:

- $Ciphertext_1 \oplus Ciphertext_2 = Plaintext_1 \oplus Plaintext_2$

The attacker now has the XOR sum of the two plaintexts. Using basic statistical analysis (like knowing the language is English, or knowing protocol headers), the attacker can easily separate them and read **both** plaintexts in cleartext, entirely bypassing the encryption without ever knowing the key.

2. The Loss of the Secret Key (AES-GCM's "Forbidden Attack")

If the algorithm is using **AES-GCM** (the most widely used AEAD cipher in the world, used in TLS/HTTPS), nonce reuse does something even worse: **It leaks the secret authentication key.**

AES-GCM provides authenticity by appending a "Tag" to the ciphertext. This tag is calculated using a polynomial equation in a mathematical space called a Galois Field.
The equation uses a secret **Authentication Subkey (called H)**, which is mathematically derived from the main master key. The final step of creating the tag is masking it with a value derived from the **Nonce**.

If you reuse the nonce, you reuse the mask.
If an attacker captures two messages with the same nonce, they can XOR the two authentication tags together. The nonce-mask cancels out, leaving the attacker with a simple polynomial algebra equation where the *only unknown variable is the secret key* H.

Because solving polynomials is mathematically easy for computers, the attacker can instantly calculate H.

What happens when the attacker has H?
They can now forge the Authentication Tag for *any* message they want. They can write a malicious plaintext, encrypt it, and sign it with the stolen H. When the receiver gets it, the math will check out perfectly, and the receiver will believe the forged message legitimately came from Alice. The entire Authenticity and Integrity of the cryptographic system is permanently broken.

Algorithm 4

```
procedure INIT(k, R)
    n_s^R ← 0
    n_r^R ← 0
end procedure
procedure SEND(m, R)
    c ← Enc_k(n_s^R, m)
    n_s^R ← n_s^R + 1 return c
end procedure
procedure RECV(c, R)
    m ← Dec_k(n_r^R, c)
    if m = ⊥ then return ⊥
    end if
    n_r^R ← n_r^R + 1
    return m
end procedure
```

Protocol is correct but not secure.
Reflection attack can be used to stop a connection between two peers. If A sends a packet, Terrence will send it back. For example, first packet is encrypted with nonce 0 so even if A has two counters this packet can be decrypted using the other A's counter that is initiated also to 0. Now A's counters are (1, 1) and B's counters are (0, 0) so they are de-synchronized thus A and B cannot communicate anymore.

he can accept his own message because we use the same key for send and receive

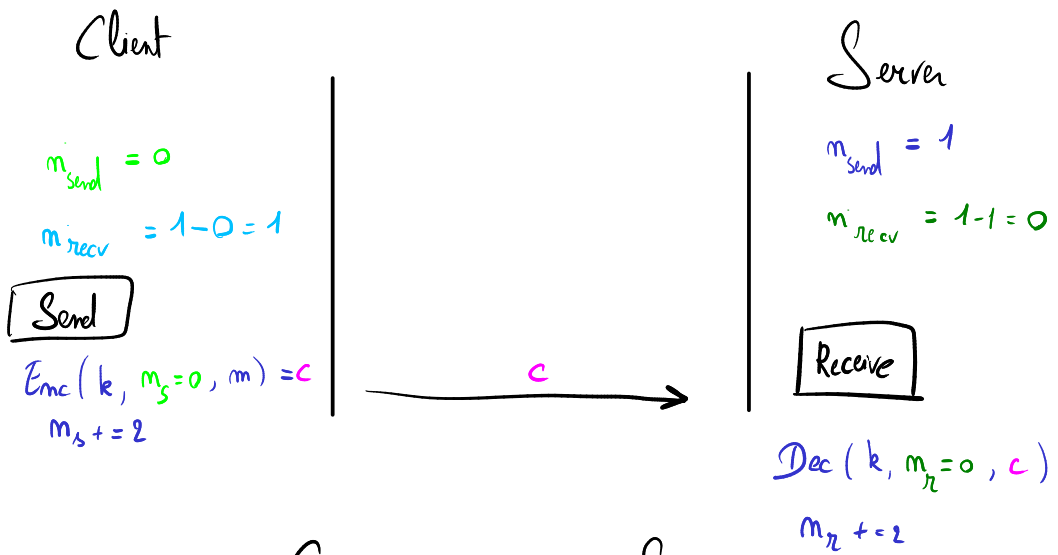
correct and secure

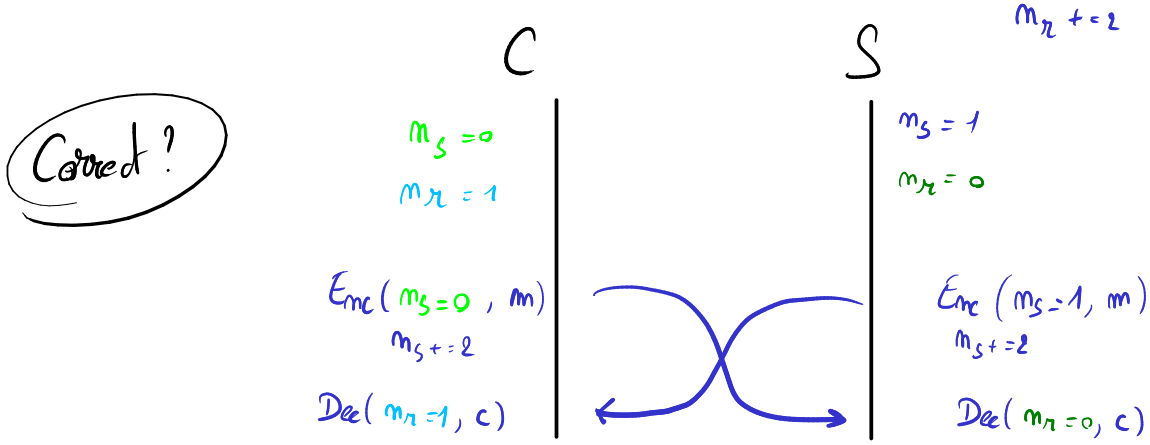
not reuse the same nonce
uniform key
accept only one nonce n the receive phase

0 = client 1 = server

Algorithm 5

procedure INIT(k, R)
  $n_s^R \leftarrow R$
  $n_r^R \leftarrow 1 - R$
end procedure
procedure SEND(m, R)
  $c \leftarrow \mathrm{Enc}_k(n_s^R, m)$
  $n_s^R \leftarrow n_s^R + 2$ **return** c
end procedure
procedure RECV(c, R)
  $m \leftarrow \mathrm{Dec}_k(n_r^R, c)$
  if $m = \perp$ **then return** $\perp$
  end if
  $n_r^R \leftarrow n_r^R + 2$ **return** m
end procedure

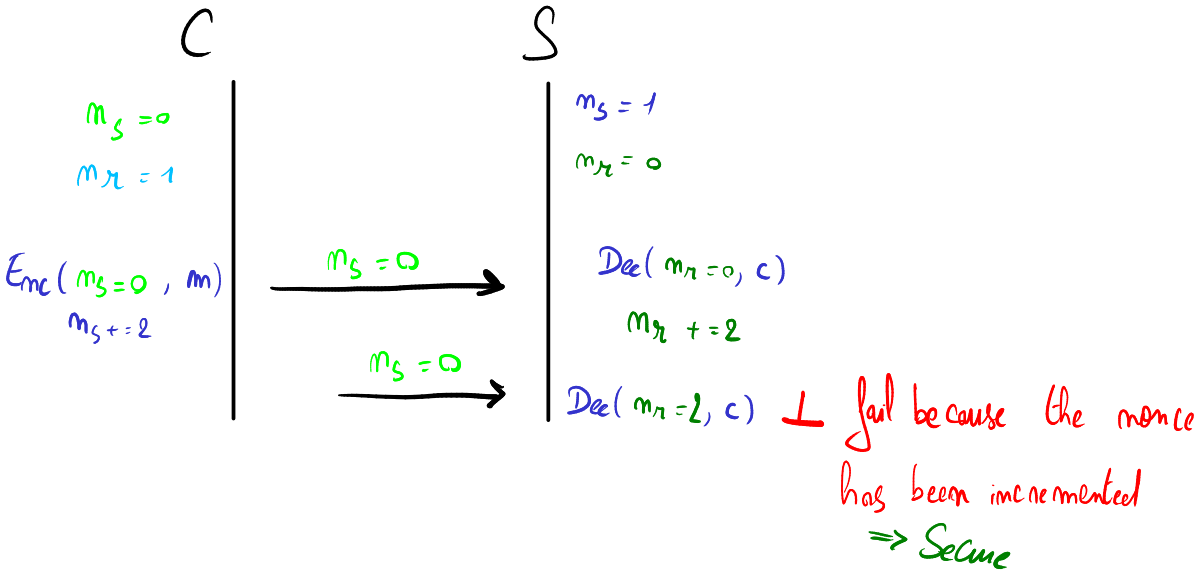




The protocol is correct, the client will always use even nonce to send (server the same even to recv) while the server will use odd number to send (clien the same to recv)

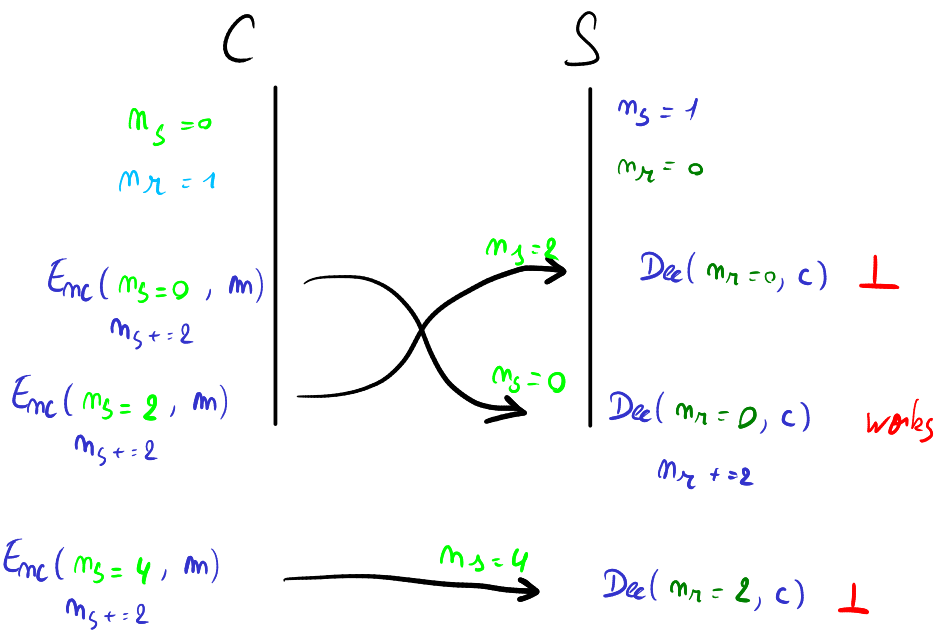
Secur ?

Replay Attack :



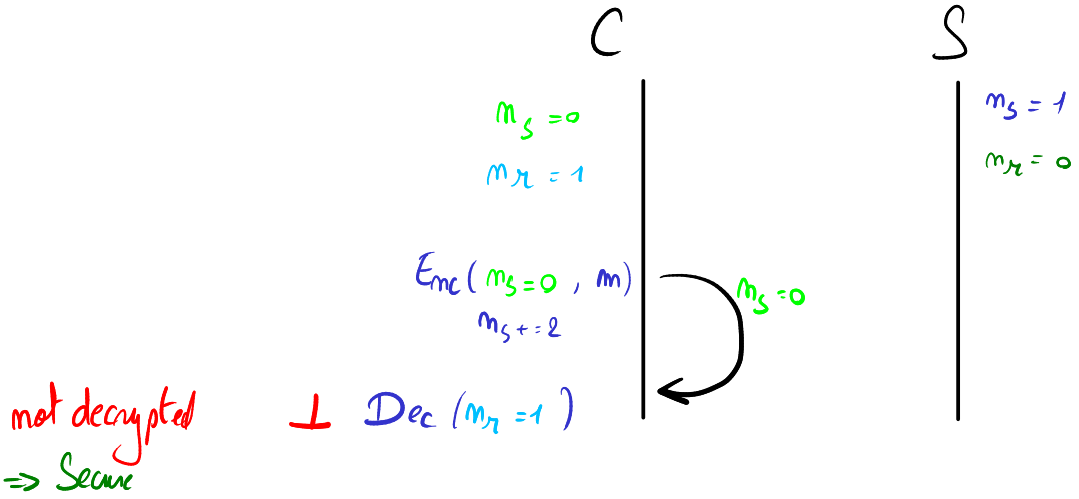
I'm not sure if its protected against Reordering attack
Unordered packet will fail to be decrypted because the nonce is incorrect

Reordering Attack :



I don't think it is secure against reordring attacks since one packet out of order will make the counter out of sync and prevent any further communication

Reflexion Attack :



Protocol correct :
- Incrementing by one or two or N doesn't matter
as long as both read and write counters are updated the same way.
- If A is R = 1 then his counters are (1, 0) on startup and his send counter has to be synchronized with B's recv counter, same thing for A's recv counter and B's send counter and B's counters with R = 0 are also (0, 1) that makes synchronization still correct. Thanks to 2 increment one counter uses even numbers and other odd numbers.

The protocol is now secure :
- Symmetric encryption assumption : no key leak.
- Double counters strategy : no re-order attack.
- Send/recv counters swift makes them never in collision : no reflection attack.

Algorithm 5

```
procedure INIT(k, R)
    n_s^R ← R
    n_r^R ← 1 − R
end procedure
procedure SEND(m, R)
    c ← Enc_k(n_s^R, m)
    n_s^R ← n_s^R + 2 return c
end procedure
procedure RECV(c, R)
    m ← Dec_k(n_r^R, c)
    if m = ⊥ then return ⊥
    end if
    n_r^R ← n_r^R + 2 return m
end procedure
```

Algorithm 6

```
procedure INIT(k, R)
    n^R ← 0
end procedure
procedure SEND(m, R)
    c ← Enc_k(n^R, m)
    n^R ← n^R + 1 return c
end procedure
procedure RECV(c, R)
    m ← Dec_k(n_r^R, m)
    if m = ⊥ then return ⊥
    end if
    n^R ← n^R + 1 return m
end procedure
```

Not even correct as algo 1.
If A sends 10 messages his n is 10 and B sends one message with nonce 0. A will try to decrypt B's message with nonce 10 that is resulting in a failed decryption.

reflexions attacks not working bc nonce incremented
reorder/replay attack are useless since the protocol is already broken

2 Confidentiality beyond standard definition

TLS traffic sniffing. Assume you are monitoring the IP traffic between a user's device and the server of a messaging app, which is encrypted with TLS.

You make the following observations: At regular intervals, a couple tiny packets are sent. At some point, one of the packets sent by the device is a bit (e.g. 200 bytes) larger than usual.

What can you conclude? In particular, assume you are running an online advertising business, and you just displayed an ad to that user.

What are the communication protocol's characteristics that are exploited in this attack? What can a protocol do in order to protect against such attacks?

variable size of packets bc lack of payload padding

Interactive SSH. When typing on a keyboard, the delay between consecutive keystrokes is highly dependent on the person typing and the characters being typed. Moreover, when using a remote shell over SSH, keystrokes need to be send with low latency, otherwise the usability is degraded. Assume a naive SSH client implementation, how is this a security issue? What is a typical sensitive piece of data interactively sent over SSH?

Analyze the countermeasure introduced by OpenSSH 9.5 using the release notes (`https://www.openssh.org/txt/release-9.5`) and the configuration description (`ObscureKeystrokeTim` in `man 5 ssh_config`). What are the trade-off of the parameters?

3 Timing Side-Channel Attack

In this exercise, you will have to perform a timing side-channel attack to retrieve a pin code. Instead of doing a simple brute force, find a way to retrieve the pin code with fewer requests. For this exercise, we propose you to build a docker to have a "black box" to query. You can also make a virtual environment or install all Python modules needed on your laptop. All the files are available in the folder `1-Timing_Attack`. For the docker, run the commands `docker build . -t timing_attack` and then `docker run -d --network host timin` The website is available at `http://localhost:5000/`.

1. As a first step, try to guess the length of the PIN, without guessing the PIN itself. Looking at the `server.py` code, how will the verification time for a PIN of the correct length compare to the time for an incorrect length?

2. As a second step, try to guess the PIN.
   - How many attempts will you need if you brute-force the PIN?
   - How many attempts will you need if you guess digits one-by-one?
   - How can you find the first digit of the PIN?
   - How can extend this attack to all digits?

3. Once your attack is successful, change the pin code and try again with your solution. It should still work even if the length changes.

4 SSH with compression security

In this exercise[1], you will use SSH with compression. All the files needed for this exercise are in the folder `2-ssh_compression`. You have a client and a server. The client compresses the data before encrypting it. As an attacker, in this scenario, you are able to ask the client to add an evil string that will be concatenated with a secret that the client sends to the server. So the client will send `evil+secret` to the server. The `secret` is composed of five capital cities chosen at random in a list and serialized into a JSON

[1] This exercise has been adapted from a lab of the 6.1600 course given at MIT (`https://61600-labs.csail.mit.edu/lab3.html`).

object. For example:

```
{
"city0": "Victoria, Seychelles",
"city1": "Seoul, South Korea",
"city2": "Paris, France",
"city3": "Vatican City, Vatican City",
"city4": "Paramaribo, Suriname",
}
```

As a first step, let us look at the behavior of compression algorithms. Using `zlib` compression (`zlib.compress` in Python), compare the length of a string of $2n$ distinct characters versus the concatenation of 2 identical strings composed of n characters (e.g. the alphabet vs twice half the alphabet). Can you extend your observations to the concatenation of `evil` and `secret`, for some well-chosen `evil`? How can you mount an attack using a compressed message length oracle?

Next, your goal is to modify `attack.py` to perform an attack and find the secret. You will need to call the function `(bytes_out, bytes_in) = client_fn(<evil_string>)`. This function causes the client to send a message to the server. It returns the number of bytes sent to the server (`bytes_out`) and the number of bytes received from the server (`bytes_in`).

For this exercise, we propose you to build a docker. You can also make a virtual environment or install all Python modules needed on your laptop. For the Docker version, you can build the Docker with `docker build . -t ssh_compression`. To launch the Docker, you can use the command `docker run --mount type=bind,src=./attack.py,dst=/app/attack` Finally, in a new terminal, connect to the docker with `docker exec -it <docker_name> /bin/bash` and execute the command `python3 main.py` to test your attack. You can change `attack.py` without rebuilding the Docker.

Finally, can you imagine an hypothetical practical setting where such an attack could be applicable (you may assume a different protocol, such as HTTPS)? Which adversary could be interested in recovering a list of cities (or another list of words)? How could an adversary have control of an `evil string`?

5 Performance comparison

In this exercise, you will have a look at the performance of various hashing and encryption algorithms. Use the VM if you do not have OpenSSL installed on your laptop.

To compare the performance of hashing and encryption algorithms, we will use `OpenSSL`. The VM runs with the version `1.1.1d`. The current stable version is `3.4.1` with many vulnerabilities fixed.

The command `openssl help` shows the different standard, message digest and cipher commands. The command `openssl speed <algorithm>` performs a speed test.

In the first instance, we are interested in the message digest `SHA` (Secure Hash Algorithm). Draw a graph comparing the performance of the different versions (SHA2 in its variants `sha256` and `sha512`, SHA3 in the variant `sha3-256`). Discuss your results. For

each algorithm, explain why or why not you would use it. Give a short example of the context where you could use it.

In a second step, we are interested in authenticated encryption algorithms and more specifically the AES (Advanced Encryption Standard) encryption with GCM (Galois Counter Mode). As previously, compare the `aes-128-gcm` and `aes-256-gcm` algorithms and explain in which context you should use which one.

Beyond the comparison, in which practical situations are these functions likely to be (or not be) performance bottlenecks?

Run benchmarks for asymetric cryptography algorithms, e.g. using Curve 25519 for ECDHE (X25519) and for signature (Ed25519). Would those be bottelnecks in practice?

Cryptographic engineering is undergoing a transition towards new algorithms that resist cryptanalysis by quantum computers (so-called "post-quantum cryptography"). Which of the algorithms discussed above are PQ-secure? How to the PQ algorithms recently-standardized by the NIST compare to non-PQ algorithms in terms of performance? (Performance is not only about computation time, but also about the size of the data.) Wikipedia can be a good starting point for your investigations [2].

[2] `https://en.wikipedia.org/wiki/Post-quantum_cryptography`